AI-assisted DevOps

Workshop AI Bridge a KII PEF ČZU

Co je DevOps?

- **Kultura a soubor praktik**, nikoli nástroj nebo pozice
- Tvoří most mezi vývojem (Dev) a provozem (Ops)
- Klíčový princip: *"Software nepřináší hodnotu, dokud ho někdo nepoužívá"*
- DevOps zajišťuje, že software se dostane k uživatelům **rychleje, bezpečně a kvalitně**

Analogie s restaurací

Restaurace	Software
Objednávka	Plánování
Příprava jídla	Vývoj
Kontrola kvality	Testování
Podávání	Nasazení
Zpětná vazba zákazníků	Monitoring

Kulturní principy DevOps

- **Spolupráce** – sdílená odpovědnost (veškerý kód je náš kód)
- **Zaměření na systémy a procesy** (ne na viníky)
- **Automatizace** pro opakované činnosti
- **Neustálé zlepšování**
- **Infrastruktura jako kód** – Terraform, Pulumi

DevOps Infinity Loop

```
      DEV                          OPS
+-------------------+       +-------------------+
|                   |       |                   |
|   Plan -> Code    |       | Release -> Deploy |
|    ^       |      |       |   ^          |    |
|   Test <- Build   |       | Operate <- Monitor|
|                   |       |                   |
+---------+---------+       +---------+---------+
          |      <---- 8 ---->        |
          +---------------------------+
```

- **8 fází** propojených do nekonečné smyčky

DORA metriky

DevOps Research and Assessment od Google Cloud

Měřitelný důkaz (KPI), že DevOps praktiky fungují:

1. Jak často nasazujeme
2. Čas od commitu (změn) po produkci
3. Poměr nasazení, která selhala
4. Čas obnovy, pokud se nasazení nepodaří

Od požadavku po nasazení (PMS → GIT)

```
+----------+    +----------+    +----------+    +----------+
|  TICKET  |--->|  BRANCH  |--->|  COMMIT  |--->|    PR    |
| PROJ-123 |    |feature/  |    |feat: ... |    | Review   |
+----------+    |PROJ-123- |    +----------+    +----+-----+
                |user-auth |                         |
                +----------+                    +----v-----+
+----------+    +----------+    +----------+    |  MERGE   |
| MONITOR  |<---|  DEPLOY  |<---|    CI    |<---|  to main |
+----------+    +----------+    | (testy)  |    +----------+
                                +----------+
```

Ticket ID je univerzální propojka mezi PMS a kódem (best practice)

Co je CI/CD?

Continuous Integration (CI)

- Automatický build a testování při každé aktualizaci repozitáře (push)
- Odhalení problémů co nejdříve

Continuous Delivery (CD)

- Automatizované nasazení do prostředí
- Kód je **vždy připraven k nasazení**

Continuous Deployment

- Automatické nasazení bez manuálního schválení

Klíčové pojmy CI/CD

Pojem	Význam
Stage	Logická skupina (build $\rightarrow$ test $\rightarrow$ deploy)
Runner	Stroj, na kterém job běží
Artifact	Výstup buildu sdílený mezi joby
Cache	Uložení závislostí mezi běhy
Environment	Dev / Staging / Production
Secret	Šifrovaná proměnná (API klíče, tokeny)

GitHub Actions

- Téměř 8 mil. dní v roce 2025, 23 mil. jobů denně
- Workflow definován v **YAML**

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npm ci
      - run: npm test
```

GitHub Environments a deployment

- **Environments**: dev → staging → production
- **Protection rules**: požadovaní schvalovatelé, práva na branche
- **Secrets** na úrovni repo / environment / organizace

Typická pipeline

- lint (kvalita kódu) → build → nasazení → testy

Alternativa: GitLab

- Lepší vizualizace pipeline, merge trains
- Možnost self-hostingu (GitLab CE)

Testovací pyramida

```
        /\              E2E testy (5-10%)
       /  \             Playwright, Cypress
      /----\
     /      \           Integrační (15-20%)
    /        \          Supertest, RTL
   /----------\
  /            \        Unit testy (70-80%)
 /              \       Jest, Vitest
/________________\
```

- **Čím níže, tím rychlejší a levnější**
- **Čím výše, tím blíže reálnému chování**

Unit testy

Testují jednu izolovanou funkci/metodu bez závislostí

```
expect(sum(2, 3)).toBe(5);
```

Integrační testy

Ověřují spolupráci více komponent dohromady (např. služba + databáze)

```
const user = await userService.create({ name: "Jan" });
const found = await userRepository.findById(user.id);
expect(found.name).toBe("Jan");
```

E2E (end-to-end) testy

Simulují reálného uživatele procházejícího celou aplikací

```
await page.goto("/login");
await page.fill("#email", "jan@test.cz");
await page.fill("#password", "heslo123");
await page.click("button[type=submit]");
await expect(page.locator("h1")).toHaveText("Dashboard");
```

Observabilita a monitoring

Jak poznáme, že aplikace funguje správně?

- **Metriky** – čísla v čase (kolik požadavků/s, jak rychlá odezva, kolik chyb)
- **Logy** – záznamy událostí ("uživatel se přihlásil", "platba selhala")
- **Traces** – sledování jednoho požadavku napříč službami

Příklad

Uživatel hlásí: *"Stránka se načítá 10 sekund."*

- **Metrika** ukáže: odezva API vzrostla z 200 ms na 8 s
- **Trace** ukáže: dotaz do databáze trvá 7,5 s
- **Log** ukáže: chybějící index na tabulce objednávek

Nástroje pro monitoring

- **Grafana** – vizualizace dat, dashboardy, alerty (open-source)
- **Sentry** – zachytává chyby v aplikaci a upozorní vývojáře
- **Uptime Kuma** – jednoduchý self-hosted monitoring dostupnosti
- **OpenTelemetry** – standard pro sběr dat (metriky, logy, traces) – nastavíte jednou, data posíláte kam chcete

AI nástroje pro vývoj

Tři úrovně

1. **IDE pluginy** – Copilot, JetBrains AI, Tabnine
2. **AI-native IDE** – Cursor, Antigravity
3. **CLI agenti** – Claude Code, Codex CLI

Klíčová čísla (2025/2026)

- **22 % mergenutého kódu** je napsáno AI (pouze přiznané)
- **91 %** týmů adoptovalo AI nástroje

AI code review

- **CodeRabbit** – údajně ~46% detekce bugů
  - Auto-trigger na každý PR, line-by-line komentáře, oprava na kliknutí
  - Není open-source
- **Qodo** – open-source alternativa

AI v CI/CD a bezpečnosti

Generování pipeline

- Popíšete slovně, co chcete – AI vygeneruje YAML workflow
- *"Při pushi spusť testy a deploy na staging"*

Generování testů

- AI analyzuje kód a navrhne testy, které pokryjí chybějící scénáře
- Dokáže spustit testy, opravit chyby a iterovat

Bezpečnost

- AI skenuje kód a navrhuje opravy zranitelností (např. SQL injection)
- Kontrola závislostí – upozornění na knihovny se známými problémy

Dokumentace

- AI generuje dokumentaci z kódu (README, API docs, changelog)

Spektrum AI autonomie

- Level 1: Autocomplete
- Level 2: Asistent (chat)
- Level 3: Agentický přístup (aktuální stav)
- Level 4: Plně autonomní (teoretická budoucnost)

MCP – Model Context Protocol

USB-C pro AI – standardní způsob, jak AI připojit k externím nástrojům

- Před MCP: každý nástroj potřeboval vlastní konektor
- S MCP: jeden standard, funguje všude

Příklad

AI agent přes MCP umí:

- Přečíst issue z GitHubu
- Podívat se na dashboard v Grafaně
- Vytvořit Pull Request
- Založit ticket v Jiře

...vše z jednoho chatu v přirozeném jazyce

Autonomní CI/CD – realita vs. vize

Co funguje dnes

- AI vytvoří PR z popisu problému (issue)
- Automatické opravy bezpečnostních chyb
- Generování a oprava testů
- Hromadné upgrady závislostí

Co je zatím teoretické

- AI samo rozhodne, co nasadit do produkce
- Pipeline bez jakéhokoli lidského dohledu

Rizika (čemu se vyvarovat)

- **Slepá důvěra** – AI kód vždy revidovat (někdy obsahuje zranitelnosti nebo není škálovatelný)
- **Příliš velké změny** – AI generuje rozsáhlé PR, které se těžko kontrolují
- **Bezpečnost** – agent by neměl schvalovat vlastní PR
- **Závislost** – rozumět tomu, co AI dělá, nejen kopírovat výstup

Shrnutí

1. DevOps = **kultura** + automatizace + zpětná vazba
2. **Ticket → Branch → PR → CI → Deploy → Monitor** – celý cyklus
3. AI pomáhá v **každé fázi**, ale potřebuje lidský dohled
4. Postupná důvěra: návrhy → asistence → agenti s kontrolou

Závěr

- AI mění způsob, jak vyvíjíme a provozujeme software
- Vývoj s AI vlastně přináší velmi vysokou úroveň abstrakce
- Týmy mají nástroje pro **zrychlení, zjednodušení a zefektivnění** práce
- Klíč k úspěchu: **rozumět základům** (DevOps, CI/CD, testing) a pak nad nimi stavět s AI

Praktická ukázka

Další informace

Webové stránky AI Bridge

aibridge.pef.czu.cz